

Programming Foundations

Python Basics, Software Principles, and Lift Control

Martin Benning

University College London

ENGF0034 – Design and Professional Skills



Lecture Overview

Objectives

To establish rigorous foundations in algorithmic problem solving, core Python syntax, software engineering best practices, and investigate a complex multi-agent engineering challenge.

Part I: Core Foundations

- ▶ Algorithms & Donald Knuth's Properties
- ▶ Euclid's GCD: Geometry & Rigorous Mathematical Proof
- ▶ Python's Object Model: Dynamic vs. Strong Typing
- ▶ Control Flow & Modular Arithmetic

Lecture Overview

Objectives

To establish rigorous foundations in algorithmic problem solving, core Python syntax, software engineering best practices, and investigate a complex multi-agent engineering challenge.

Part I: Core Foundations

- ▶ Algorithms & Donald Knuth's Properties
- ▶ Euclid's GCD: Geometry & Rigorous Mathematical Proof
- ▶ Python's Object Model: Dynamic vs. Strong Typing
- ▶ Control Flow & Modular Arithmetic

Part II: Engineering & Systems

- ▶ Strings in Depth (Sequences & Immutability)
- ▶ Structuring Code: Functions, Scope (LEGB)
- ▶ Defensive Programming: Tests & Exceptions
- ▶ The Lift Problem: Bunching to Destination Dispatch



Algorithms and Computation

What is an Algorithm?

Definition

An algorithm is a finite sequence of well-defined, computer-implementable instructions designed to solve a specific class of problems or perform a computation.

What is an Algorithm?

Definition

An algorithm is a finite sequence of well-defined, computer-implementable instructions designed to solve a specific class of problems or perform a computation.

The Two Pillars of Analysis

1. **Correctness:** Does the algorithm provably yield the correct output for all valid inputs?
2. **Efficiency:** How do execution time and memory requirements scale with input size N ?

What is an Algorithm?

Definition

An algorithm is a finite sequence of well-defined, computer-implementable instructions designed to solve a specific class of problems or perform a computation.

The Two Pillars of Analysis

1. **Correctness:** Does the algorithm provably yield the correct output for all valid inputs?
2. **Efficiency:** How do execution time and memory requirements scale with input size N ?

Key Criteria (Donald Knuth)

- ▶ **Finiteness:** Must terminate after a finite number of steps for all inputs.

What is an Algorithm?

Definition

An algorithm is a finite sequence of well-defined, computer-implementable instructions designed to solve a specific class of problems or perform a computation.

The Two Pillars of Analysis

1. **Correctness:** Does the algorithm provably yield the correct output for all valid inputs?
2. **Efficiency:** How do execution time and memory requirements scale with input size N ?

Key Criteria (Donald Knuth)

- ▶ **Finiteness:** Must terminate after a finite number of steps for all inputs.
- ▶ **Definiteness:** Every step must be unambiguous and precisely defined.

What is an Algorithm?

Definition

An algorithm is a finite sequence of well-defined, computer-implementable instructions designed to solve a specific class of problems or perform a computation.

The Two Pillars of Analysis

1. **Correctness:** Does the algorithm provably yield the correct output for all valid inputs?
2. **Efficiency:** How do execution time and memory requirements scale with input size N ?

Key Criteria (Donald Knuth)

- ▶ **Finiteness:** Must terminate after a finite number of steps for all inputs.
- ▶ **Definiteness:** Every step must be unambiguous and precisely defined.
- ▶ **Inputs & Outputs:** Zero or more inputs; one or more defined outputs.

What is an Algorithm?

Definition

An algorithm is a finite sequence of well-defined, computer-implementable instructions designed to solve a specific class of problems or perform a computation.

The Two Pillars of Analysis

1. **Correctness:** Does the algorithm provably yield the correct output for all valid inputs?
2. **Efficiency:** How do execution time and memory requirements scale with input size N ?

Key Criteria (Donald Knuth)

- ▶ **Finiteness:** Must terminate after a finite number of steps for all inputs.
- ▶ **Definiteness:** Every step must be unambiguous and precisely defined.
- ▶ **Inputs & Outputs:** Zero or more inputs; one or more defined outputs.
- ▶ **Effectiveness:** Operations must be sufficiently basic to execute in finite time.

Algorithm of the Week: Euclid's GCD

Motivation and Cryptographic Relevance

Euclid of Alexandria (c. 300 BC)

One of the oldest non-trivial algorithms in continuous use (*Elements*, Book VII).

Algorithm of the Week: Euclid's GCD

Motivation and Cryptographic Relevance

Euclid of Alexandria (c. 300 BC)

One of the oldest non-trivial algorithms in continuous use (*Elements*, Book VII).

- The **Greatest Common Divisor (GCD)** is the largest positive integer that divides both a and b without remainder.

Algorithm of the Week: Euclid's GCD

Motivation and Cryptographic Relevance

Euclid of Alexandria (c. 300 BC)

One of the oldest non-trivial algorithms in continuous use (*Elements*, Book VII).

- ▶ The **Greatest Common Divisor (GCD)** is the largest positive integer that divides both a and b without remainder.
- ▶ Essential for simplifying fractions:

$$\frac{390253}{228769} = \frac{29 \cdot 13457}{17 \cdot 13457} = \frac{29}{17}$$

since $\text{gcd}(390253, 228769) = 13457$.



Algorithm of the Week: Euclid's GCD

Motivation and Cryptographic Relevance

Euclid of Alexandria (c. 300 BC)

One of the oldest non-trivial algorithms in continuous use (*Elements*, Book VII).

- ▶ The **Greatest Common Divisor (GCD)** is the largest positive integer that divides both a and b without remainder.
- ▶ Essential for simplifying fractions:

$$\frac{390253}{228769} = \frac{29 \cdot 13457}{17 \cdot 13457} = \frac{29}{17}$$

since $\text{gcd}(390253, 228769) = 13457$.

The Computational Challenge

Naive approach: Compute prime factorisations.

- ▶ Finding prime factors of large numbers has no known classical polynomial-time algorithm.



Algorithm of the Week: Euclid's GCD

Motivation and Cryptographic Relevance

Euclid of Alexandria (c. 300 BC)

One of the oldest non-trivial algorithms in continuous use (*Elements*, Book VII).

- ▶ The **Greatest Common Divisor (GCD)** is the largest positive integer that divides both a and b without remainder.
- ▶ Essential for simplifying fractions:

$$\frac{390253}{228769} = \frac{29 \cdot 13457}{17 \cdot 13457} = \frac{29}{17}$$

since $\text{gcd}(390253, 228769) = 13457$.

The Computational Challenge

Naive approach: Compute prime factorisations.

- ▶ Finding prime factors of large numbers has no known classical polynomial-time algorithm.
- ▶ This computational hardness is the basis of modern **RSA public-key cryptography!**



Algorithm of the Week: Euclid's GCD

Motivation and Cryptographic Relevance

Euclid of Alexandria (c. 300 BC)

One of the oldest non-trivial algorithms in continuous use (*Elements*, Book VII).

- ▶ The **Greatest Common Divisor (GCD)** is the largest positive integer that divides both a and b without remainder.
- ▶ Essential for simplifying fractions:

$$\frac{390253}{228769} = \frac{29 \cdot 13457}{17 \cdot 13457} = \frac{29}{17}$$

since $\text{gcd}(390253, 228769) = 13457$.

The Computational Challenge

Naive approach: Compute prime factorisations.

- ▶ Finding prime factors of large numbers has no known classical polynomial-time algorithm.
- ▶ This computational hardness is the basis of modern **RSA public-key cryptography**!
- ▶ Euclid's algorithm finds $\text{gcd}(a, b)$ without factoring either number!



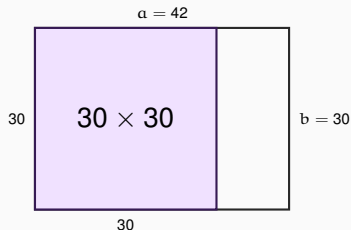
Euclid's Algorithm: Geometric Tiling

The Fundamental Invariant

For positive integers $a > b$:
 $\gcd(a, b) = \gcd(a - b, b)$

Geometric View (Tiling a Rectangle):

- ▶ Tiling an $a \times b$ floor with square tiles of side g .
- ▶ Example 42×30 :
 1. Cut off largest square: 30×30 . Remaining: 12×30 .



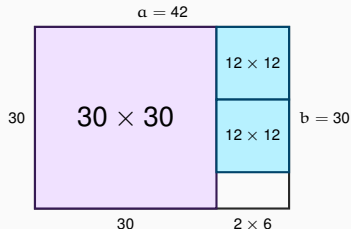
Euclid's Algorithm: Geometric Tiling

The Fundamental Invariant

For positive integers $a > b$:
 $\gcd(a, b) = \gcd(a - b, b)$

Geometric View (Tiling a Rectangle):

- ▶ Tiling an $a \times b$ floor with square tiles of side g .
- ▶ Example 42×30 :
 1. Cut off largest square: 30×30 . Remaining: 12×30 .
 2. Cut off two 12×12 squares. Remaining: 12×6 .



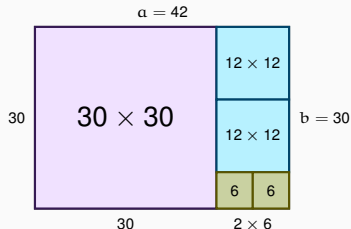
Euclid's Algorithm: Geometric Tiling

The Fundamental Invariant

For positive integers $a > b$:
 $\gcd(a, b) = \gcd(a - b, b)$

Geometric View (Tiling a Rectangle):

- ▶ Tiling an $a \times b$ floor with square tiles of side g .
- ▶ Example 42×30 :
 1. Cut off largest square: 30×30 . Remaining: 12×30 .
 2. Cut off two 12×12 squares. Remaining: 12×6 .
 3. Cut off two 6×6 squares. Remaining: 0!
- ▶ Tile fitting with zero remainder is
 $6 \implies \gcd(42, 30) = 6$.



State Transition Sequence

$(42, 30) \xrightarrow{-30} (12, 30) \xrightarrow{-12} (12, 18) \xrightarrow{-12} (12, 6) \xrightarrow{-6} (6, 6)$

Mathematical Rigour: Proof of Euclid's Algorithm (I)

Theorem 1: Guaranteed Termination via Well-Ordering

Theorem 1: Termination

For any positive integers $a, b \in \mathbb{Z}^+$, Euclid's subtraction algorithm terminates in a finite number of iterations.

Mathematical Rigour: Proof of Euclid's Algorithm (I)

Theorem 1: Guaranteed Termination via Well-Ordering

Theorem 1: Termination

For any positive integers $a, b \in \mathbb{Z}^+$, Euclid's subtraction algorithm terminates in a finite number of iterations.

Potential Function (Invariant)

Define $S(a, b) = a + b$. Since $a, b \geq 1$, we have $S \geq 2$, bounded strictly below by zero in $\mathbb{Z}^+$.

Mathematical Rigour: Proof of Euclid's Algorithm (I)

Theorem 1: Guaranteed Termination via Well-Ordering

Theorem 1: Termination

For any positive integers $a, b \in \mathbb{Z}^+$, Euclid's subtraction algorithm terminates in a finite number of iterations.

Potential Function (Invariant)

Define $S(a, b) = a + b$. Since $a, b \geq 1$, we have $S \geq 2$, bounded strictly below by zero in $\mathbb{Z}^+$.

Proof of Strict Descent

In each iteration where $a \neq b$:

- ▶ If $a > b$: new state $(a - b, b)$ has sum $S' = (a - b) + b = a < a + b = S$.

Mathematical Rigour: Proof of Euclid's Algorithm (I)

Theorem 1: Guaranteed Termination via Well-Ordering

Theorem 1: Termination

For any positive integers $a, b \in \mathbb{Z}^+$, Euclid's subtraction algorithm terminates in a finite number of iterations.

Potential Function (Invariant)

Define $S(a, b) = a + b$. Since $a, b \geq 1$, we have $S \geq 2$, bounded strictly below by zero in $\mathbb{Z}^+$.

Proof of Strict Descent

In each iteration where $a \neq b$:

- ▶ If $a > b$: new state $(a - b, b)$ has sum $S' = (a - b) + b = a < a + b = S$.
- ▶ If $b > a$: new state $(a, b - a)$ has sum $S' = a + (b - a) = b < a + b = S$.

Mathematical Rigour: Proof of Euclid's Algorithm (I)

Theorem 1: Guaranteed Termination via Well-Ordering

Theorem 1: Termination

For any positive integers $a, b \in \mathbb{Z}^+$, Euclid's subtraction algorithm terminates in a finite number of iterations.

Potential Function (Invariant)

Define $S(a, b) = a + b$. Since $a, b \geq 1$, we have $S \geq 2$, bounded strictly below by zero in $\mathbb{Z}^+$.

Proof of Strict Descent

In each iteration where $a \neq b$:

- ▶ If $a > b$: new state $(a - b, b)$ has sum $S' = (a - b) + b = a < a + b = S$.
- ▶ If $b > a$: new state $(a, b - a)$ has sum $S' = a + (b - a) = b < a + b = S$.

The sum sequence strictly decreases in $\mathbb{Z}^+$: $S_0 > S_1 > S_2 > \dots \geq 2$. By the **Well-Ordering Principle**, an infinite descending chain of positive integers is impossible $\implies$ algorithm terminates with $a = b$. □



Mathematical Rigour: Proof of Euclid's Algorithm (II)

Theorem 2: Invariance of the GCD via Contradiction

Theorem 2: Invariance of the GCD

If $a, b \in \mathbb{Z}^+$ with $a > b$, then
 $\gcd(a, b) = \gcd(a - b, b)$.

Mathematical Rigour: Proof of Euclid's Algorithm (II)

Theorem 2: Invariance of the GCD via Contradiction

Theorem 2: Invariance of the GCD

If $a, b \in \mathbb{Z}^+$ with $a > b$, then
 $\gcd(a, b) = \gcd(a - b, b)$.

Setup:

► Let $g = \gcd(a, b)$. By definition:

$$a = \alpha g, \quad b = \beta g \quad \text{with } \gcd(\alpha, \beta) = 1$$

where α, β are coprime integers.

Mathematical Rigour: Proof of Euclid's Algorithm (II)

Theorem 2: Invariance of the GCD via Contradiction

Theorem 2: Invariance of the GCD

If $a, b \in \mathbb{Z}^+$ with $a > b$, then
 $\gcd(a, b) = \gcd(a - b, b)$.

Setup:

- ▶ Let $g = \gcd(a, b)$. By definition:

$$a = \alpha g, \quad b = \beta g \quad \text{with } \gcd(\alpha, \beta) = 1$$

where α, β are coprime integers.

- ▶ After subtraction: $a' = a - b = (\alpha - \beta)g$.
- ▶ Clearly, g divides both a' and b . Is it the *greatest*?

Mathematical Rigour: Proof of Euclid's Algorithm (II)

Theorem 2: Invariance of the GCD via Contradiction

Theorem 2: Invariance of the GCD

If $a, b \in \mathbb{Z}^+$ with $a > b$, then
 $\gcd(a, b) = \gcd(a - b, b)$.

Setup:

- ▶ Let $g = \gcd(a, b)$. By definition:

$$a = \alpha g, \quad b = \beta g \quad \text{with } \gcd(\alpha, \beta) = 1$$

where α, β are coprime integers.

- ▶ After subtraction: $a' = a - b = (\alpha - \beta)g$.
- ▶ Clearly, g divides both a' and b . Is it the *greatest*?

Proof by Contradiction

1. Assume $\gcd(a', b) = g' > g$.

Mathematical Rigour: Proof of Euclid's Algorithm (II)

Theorem 2: Invariance of the GCD via Contradiction

Theorem 2: Invariance of the GCD

If $a, b \in \mathbb{Z}^+$ with $a > b$, then
 $\gcd(a, b) = \gcd(a - b, b)$.

Setup:

- ▶ Let $g = \gcd(a, b)$. By definition:

$$a = \alpha g, \quad b = \beta g \quad \text{with } \gcd(\alpha, \beta) = 1$$

where α, β are coprime integers.

- ▶ After subtraction: $a' = a - b = (\alpha - \beta)g$.
- ▶ Clearly, g divides both a' and b . Is it the *greatest*?

Proof by Contradiction

1. Assume $\gcd(a', b) = g' > g$.
2. Then $\alpha - \beta$ and β must share a factor $c > 1$:
 $c \mid (\alpha - \beta)$ and $c \mid \beta$.

Mathematical Rigour: Proof of Euclid's Algorithm (II)

Theorem 2: Invariance of the GCD via Contradiction

Theorem 2: Invariance of the GCD

If $a, b \in \mathbb{Z}^+$ with $a > b$, then
 $\gcd(a, b) = \gcd(a - b, b)$.

Setup:

- ▶ Let $g = \gcd(a, b)$. By definition:

$$a = \alpha g, \quad b = \beta g \quad \text{with } \gcd(\alpha, \beta) = 1$$

where α, β are coprime integers.

- ▶ After subtraction: $a' = a - b = (\alpha - \beta)g$.
- ▶ Clearly, g divides both a' and b . Is it the *greatest*?

Proof by Contradiction

1. Assume $\gcd(a', b) = g' > g$.
2. Then $\alpha - \beta$ and β must share a factor $c > 1$:
 $c \mid (\alpha - \beta)$ and $c \mid \beta$.
3. If c divides both, it must divide their sum:

$$c \mid ((\alpha - \beta) + \beta) \implies c \mid \alpha$$

Mathematical Rigour: Proof of Euclid's Algorithm (II)

Theorem 2: Invariance of the GCD via Contradiction

Theorem 2: Invariance of the GCD

If $a, b \in \mathbb{Z}^+$ with $a > b$, then
 $\gcd(a, b) = \gcd(a - b, b)$.

Setup:

- ▶ Let $g = \gcd(a, b)$. By definition:

$$a = \alpha g, \quad b = \beta g \quad \text{with } \gcd(\alpha, \beta) = 1$$

where α, β are coprime integers.

- ▶ After subtraction: $a' = a - b = (\alpha - \beta)g$.
- ▶ Clearly, g divides both a' and b . Is it the *greatest*?

Proof by Contradiction

1. Assume $\gcd(a', b) = g' > g$.
2. Then $\alpha - \beta$ and β must share a factor $c > 1$:
 $c \mid (\alpha - \beta)$ and $c \mid \beta$.

3. If c divides both, it must divide their sum:

$$c \mid ((\alpha - \beta) + \beta) \implies c \mid \alpha$$

4. Hence $c > 1$ divides both α and β .

Mathematical Rigour: Proof of Euclid's Algorithm (II)

Theorem 2: Invariance of the GCD via Contradiction

Theorem 2: Invariance of the GCD

If $a, b \in \mathbb{Z}^+$ with $a > b$, then
 $\gcd(a, b) = \gcd(a - b, b)$.

Setup:

- ▶ Let $g = \gcd(a, b)$. By definition:

$$a = \alpha g, \quad b = \beta g \quad \text{with } \gcd(\alpha, \beta) = 1$$

where α, β are coprime integers.

- ▶ After subtraction: $a' = a - b = (\alpha - \beta)g$.
- ▶ Clearly, g divides both a' and b . Is it the *greatest*?

Proof by Contradiction

1. Assume $\gcd(a', b) = g' > g$.
2. Then $\alpha - \beta$ and β must share a factor $c > 1$:
 $c \mid (\alpha - \beta)$ and $c \mid \beta$.
3. If c divides both, it must divide their sum:

$$c \mid ((\alpha - \beta) + \beta) \implies c \mid \alpha$$

4. Hence $c > 1$ divides both α and β .
5. **Contradiction!** This violates $\gcd(\alpha, \beta) = 1$.

Mathematical Rigour: Proof of Euclid's Algorithm (II)

Theorem 2: Invariance of the GCD via Contradiction

Theorem 2: Invariance of the GCD

If $a, b \in \mathbb{Z}^+$ with $a > b$, then
 $\gcd(a, b) = \gcd(a - b, b)$.

Setup:

- ▶ Let $g = \gcd(a, b)$. By definition:

$$a = \alpha g, \quad b = \beta g \quad \text{with } \gcd(\alpha, \beta) = 1$$

where α, β are coprime integers.

- ▶ After subtraction: $a' = a - b = (\alpha - \beta)g$.
- ▶ Clearly, g divides both a' and b . Is it the *greatest*?

Proof by Contradiction

1. Assume $\gcd(a', b) = g' > g$.
2. Then $\alpha - \beta$ and β must share a factor $c > 1$:
 $c \mid (\alpha - \beta)$ and $c \mid \beta$.
3. If c divides both, it must divide their sum:

$$c \mid ((\alpha - \beta) + \beta) \implies c \mid \alpha$$

4. Hence $c > 1$ divides both α and β .
5. **Contradiction!** This violates $\gcd(\alpha, \beta) = 1$.
6. Therefore, $g' = g \implies \gcd(a - b, b) = g$. $\square$

Optimisation: Subtraction vs. Modulo

Subtraction-based Form

Repeatedly subtracts smaller from larger.

- ▶ Simple to state and prove.
- ▶ Worst case: $a \gg b$ (e.g., $a = 10^9$, $b = 1$).
- ▶ Requires 10^9 subtractions!
- ▶ **Time complexity:** $O(\max(a, b))$.

Optimisation: Subtraction vs. Modulo

Subtraction-based Form

Repeatedly subtracts smaller from larger.

- ▶ Simple to state and prove.
- ▶ Worst case: $a \gg b$ (e.g., $a = 10^9$, $b = 1$).
- ▶ Requires 10^9 subtractions!
- ▶ **Time complexity:** $O(\max(a, b))$.

Modulo-based Form (Standard Euclid)

Replaces repeated subtraction with remainder:

$$\gcd(a, b) = \gcd(b, a \bmod b) \quad (b > 0)$$

- ▶ Collapses multiple subtractions into 1 division.
- ▶ **Lamé's Theorem:** At most $5 \times \text{digits}(b)$ steps.
- ▶ **Time complexity:** $O(\log(\min(a, b)))$.

Optimisation: Subtraction vs. Modulo

Subtraction-based Form

Repeatedly subtracts smaller from larger.

- ▶ Simple to state and prove.
- ▶ Worst case: $a \gg b$ (e.g., $a = 10^9$, $b = 1$).
- ▶ Requires 10^9 subtractions!
- ▶ **Time complexity:** $O(\max(a, b))$.

Modulo-based Form (Standard Euclid)

Replaces repeated subtraction with remainder:

$$\gcd(a, b) = \gcd(b, a \bmod b) \quad (b > 0)$$

- ▶ Collapses multiple subtractions into 1 division.
- ▶ **Lamé's Theorem:** At most $5 \times \text{digits}(b)$ steps.
- ▶ **Time complexity:** $O(\log(\min(a, b)))$.

Key Takeaway: Both forms preserve the exact same invariant, but modular arithmetic drops runtime from linear to logarithmic!



Python Foundations

From Algorithm to Program: Syntax and Idioms

Core Components

- ▶ **Expression:** Fragment that evaluates to a value (e.g., 42, $a - b$).
- ▶ **Statement:** Executable action (e.g., assignment, condition, loop).
- ▶ **Assignment (=):** Binds a name on the LHS to an object evaluated on the RHS.

```
1  # Standard step-by-step assignment
2  a = 42
3  b = 30
4  a = a - b  # RHS evaluated first,
              then rebound
```

From Algorithm to Program: Syntax and Idioms

Core Components

- ▶ **Expression:** Fragment that evaluates to a value (e.g., 42, $a - b$).
- ▶ **Statement:** Executable action (e.g., assignment, condition, loop).
- ▶ **Assignment (=):** Binds a name on the LHS to an object evaluated on the RHS.

```
1 # Standard step-by-step assignment
2 a = 42
3 b = 30
4 a = a - b # RHS evaluated first,
            then rebound
```

Python Idiom: Simultaneous Assignment

Python evaluates all RHS expressions into a tuple before binding any LHS names.

```
1 # Simultaneous initialisation
2 a, b = 42, 30
3
4 # Elegant swap without temporary
   variable!
5 a, b = b, a
```

- ▶ No need for a temp variable.
- ▶ Eliminates intermediate assignment bugs.

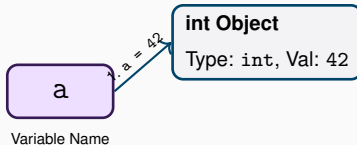


Python's Object Model: Dynamic vs. Strong Typing

1. Dynamic Typing

Variables in Python are **names (references)**, not typed memory boxes.

- ▶ Types belong to *objects*, not variables.
- ▶ A name can refer to an integer, then later a string.

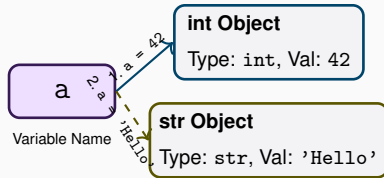


Python's Object Model: Dynamic vs. Strong Typing

1. Dynamic Typing

Variables in Python are **names (references)**, not typed memory boxes.

- ▶ Types belong to *objects*, not variables.
- ▶ A name can refer to an integer, then later a string.

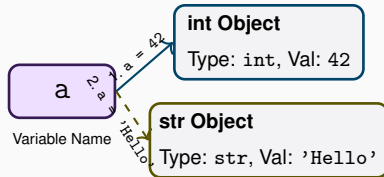


Python's Object Model: Dynamic vs. Strong Typing

1. Dynamic Typing

Variables in Python are **names (references)**, not typed memory boxes.

- ▶ Types belong to *objects*, not variables.
- ▶ A name can refer to an integer, then later a string.



2. Strong Typing

Python strictly enforces type safety at runtime.

- ▶ No implicit conversions between incompatible types!
- ▶ "Age: " + 20 → **TypeError**.
- ▶ Must explicitly cast: "Age: " + str(20).

Contrast with C / Java

In statically typed languages, variables are fixed-type containers: `int a = 42;` cannot hold a string.

Type Systems and Modern Python: Type Hints

Static vs. Dynamic Typing

- ▶ **Static (C++, Java, Rust):** Types verified at compile time. Catches errors early, optimal performance.
- ▶ **Dynamic (Python, Ruby):** Types verified at runtime. High flexibility, rapid prototyping.

Type Systems and Modern Python: Type Hints

Static vs. Dynamic Typing

- ▶ **Static (C++, Java, Rust):** Types verified at compile time. Catches errors early, optimal performance.
- ▶ **Dynamic (Python, Ruby):** Types verified at runtime. High flexibility, rapid prototyping.

Lightweight Formal Methods

Type checking prevents invalid operations (e.g., dividing text) before they corrupt system state.

Modern Python: Type Hints (PEP 484)

Python supports optional type annotations:

```
1 def gcd(a: int, b: int) -> int:
2     """Compute GCD of two positive
3         integers."""
4     while a != b:
5         if a > b:
6             a -= b
7         else:
8             b -= a
9     return a
```

- ▶ Documents explicit interface contracts.
- ▶ Static type checkers (mypy) catch bugs before runtime.



Numeric Types and Division Semantics

Arbitrary Precision (`int`)

Python integers never overflow 64 bits:

```
>>> 2 ** 100  
1267650600228229401496703205376
```

Numeric Types and Division Semantics

Arbitrary Precision (int)

Python integers never overflow 64 bits:

```
>>> 2 ** 100
1267650600228229401496703205376
```

Division Operators

- ▶ True division / **always** returns a float: $6 / 2 \rightarrow 3.0$
- ▶ Floor division // truncates to integer: $7 // 2 \rightarrow 3$
- ▶ Modulo % returns integer remainder: $7 \% 2 \rightarrow 1$

Application to Assignment 1

- ▶ **Exercise 2 (Odd/Even):** Check parity:

```
1 if n % 2 == 0:
2     print("Even")
3 else:
4     print("Odd")
```

- ▶ **Exercise 3 (FizzBuzz):** Multiples with % 3 & % 5.
- ▶ **Exercise 4 (Pi):** True division / with coordinates.

Concept Check: Types and Operators

Interactive Question (30 seconds)

What are the values and types of x, y, and z in:

```
1 x = 7 // 2
2 y = 7 / 2
3 z = 7 % 2
```

Options:

- A) x=3 (int), y=3 (int), z=1 (int)
- B) x=3.5 (float), y=3.5 (float), z=1 (int)
- C) x=3 (int), y=3.5 (float), z=1 (int)
- D) x=3 (int), y=3.5 (float), z=0.5 (float)

Concept Check: Types and Operators

Interactive Question (30 seconds)

What are the values and types of x, y, and z in:

```
1 x = 7 // 2  
2 y = 7 / 2  
3 z = 7 % 2
```

Options:

- A) x=3 (int), y=3 (int), z=1 (int)
- B) x=3.5 (float), y=3.5 (float), z=1 (int)
- C) x=3 (int), y=3.5 (float), z=1 (int)
- D) x=3 (int), y=3.5 (float), z=0.5 (float)

Answer: C

- ▶ // computes floor quotient: 3.
- ▶ / computes true float quotient: 3.5.
- ▶ % computes integer remainder: 1.

Remember: In Python 3, / **always** produces a float!

Control Flow: Conditionals and Loops

Selection: if / elif / else

Branches based on boolean expressions
(bool: True / False).

```
1  if condition1:
2      action_1()
3  elif condition2:
4      action_2()
5  else:
6      fallback_action()
```

- ▶ Comparisons: ==, !=, <, >, <=, >=.
- ▶ Logical: and, or, not (short-circuit).
- ▶ **Semantic Indentation:** 4 spaces define blocks.

Control Flow: Conditionals and Loops

Selection: if / elif / else

Branches based on boolean expressions (bool: True / False).

```
1  if condition1:
2      action_1()
3  elif condition2:
4      action_2()
5  else:
6      fallback_action()
```

- ▶ Comparisons: ==, !=, <, >, <=, >=.
- ▶ Logical: and, or, not (short-circuit).
- ▶ **Semantic Indentation:** 4 spaces define blocks.

Repetition: The while Loop

Repeats execution while condition is True:

```
1  while condition:
2      # Loop body
3      update_state()
```

Control Flow: Conditionals and Loops

Selection: if / elif / else

Branches based on boolean expressions (bool: True / False).

```
1  if condition1:
2      action_1()
3  elif condition2:
4      action_2()
5  else:
6      fallback_action()
```

- ▶ Comparisons: ==, !=, <, >, <=, >=.
- ▶ Logical: and, or, not (short-circuit).
- ▶ **Semantic Indentation:** 4 spaces define blocks.

Repetition: The while Loop

Repeats execution while condition is True:

```
1  while condition:
2      # Loop body
3      update_state()
```

Guaranteeing Termination

Every iteration must update variables towards making the condition False. Otherwise, an **infinite loop** occurs!

Automating Euclid's Algorithm in Python

Python Implementation

```
1  # Initial values
2  a, b = 42, 30
3
4  # Loop until values converge to GCD
5  while a != b:
6      if a > b:
7          a = a - b
8      else:
9          b = b - a
10
11  print(f"The GCD is: {a}") # Output: 6
```

Automating Euclid's Algorithm in Python

Python Implementation

```
1  # Initial values
2  a, b = 42, 30
3
4  # Loop until values converge to GCD
5  while a != b:
6      if a > b:
7          a = a - b
8      else:
9          b = b - a
10
11 print(f"The GCD is: {a}") # Output: 6
```

Iter	Condition	Update	State (a, b)
0	$42 \neq 30$	$a \leftarrow 42 - 30$	(12, 30)
1	$12 \neq 30$	$b \leftarrow 30 - 12$	(12, 18)
2	$12 \neq 18$	$b \leftarrow 18 - 12$	(12, 6)
3	$12 \neq 6$	$a \leftarrow 12 - 6$	(6, 6)
4	$6 \neq 6$ (False)	Terminate	GCD = 6

Automating Euclid's Algorithm in Python

Python Implementation

```
1  # Initial values
2  a, b = 42, 30
3
4  # Loop until values converge to GCD
5  while a != b:
6      if a > b:
7          a = a - b
8      else:
9          b = b - a
10
11 print(f"The GCD is: {a}") # Output: 6
```

Iter	Condition	Update	State (a, b)
0	$42 \neq 30$	$a \leftarrow 42 - 30$	(12, 30)
1	$12 \neq 30$	$b \leftarrow 30 - 12$	(12, 18)
2	$12 \neq 18$	$b \leftarrow 18 - 12$	(12, 6)
3	$12 \neq 6$	$a \leftarrow 12 - 6$	(6, 6)
4	$6 \neq 6$ (False)	Terminate	GCD = 6

Invariant Preserved

$\text{gcd}(42, 30) = \text{gcd}(12, 30) = \text{gcd}(12, 18) = \text{gcd}(12, 6) = \text{gcd}(6, 6) = 6$

Strings in Depth

Strings (str) as Sequences

Definition

In Python 3, a string is an **immutable sequence of Unicode code points** (encoded as UTF-8 in source files), supporting text and symbols worldwide.

String Literals

```
1 s1 = 'Single quotes'
2 s2 = "Double quotes"
3 s3 = """Multi-line string
4 spanning multiple lines"""
```


Strings (str) as Sequences

Definition

In Python 3, a string is an **immutable sequence of Unicode code points** (encoded as UTF-8 in source files), supporting text and symbols worldwide.

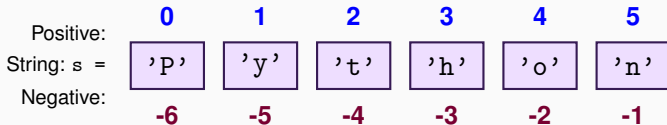
String Literals

```
1 s1 = 'Single quotes'
2 s2 = "Double quotes"
3 s3 = """Multi-line string
4 spanning multiple lines"""
```

Sequence Properties

- ▶ Length: `len(s)` returns character count.
- ▶ Concatenation: `s1 + s2`.
- ▶ Repetition: `"Ha" * 3` → `"HaHaHa"`.
- ▶ Membership: `'th' in 'Python'` → `True`.

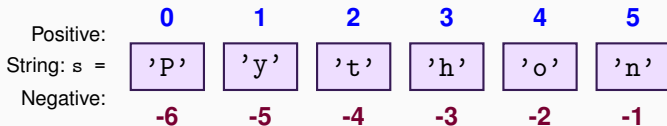
String Indexing and Slicing



Indexing (Zero-based)

```
1 s = "Python"
2 s[0]    # 'P' (first character)
3 s[-1]   # 'n' (last character)
```

String Indexing and Slicing



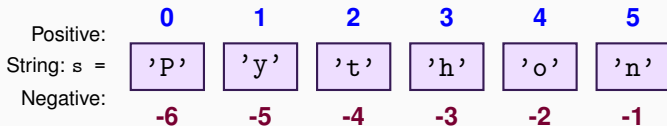
Indexing (Zero-based)

```
1 s = "Python"
2 s[0]    # 'P' (first character)
3 s[-1]   # 'n' (last character)
```

Slicing: s[start:stop:step]

```
1 s[0:2]   # 'Py' (stop is exclusive!)
2 s[2:]    # 'thon' (index 2 to end)
3 s[::-1]  # 'nohtyP' (reverses string!)
```

String Indexing and Slicing



Indexing (Zero-based)

```
1 s = "Python"
2 s[0]    # 'P' (first character)
3 s[-1]   # 'n' (last character)
```

Slicing: s[start:stop:step]

```
1 s[0:2]   # 'Py' (stop is exclusive!)
2 s[2:]    # 'thon' (index 2 to end)
3 s[::-1]  # 'nohtyP' (reverses string!)
```

Coursework Note: String indexing and slicing are central to **Exercise 5 (Caesar Cipher)** in Assignment 1.

String Immutability: Concept and Performance

Key Concept: Strings are Immutable

Once created in memory, string objects cannot be modified in place.

```
1 s = "Python"  
2 s[0] = "J"    # TypeError: 'str' object does not support item assignment
```

Creating New Strings

To change a string, construct a new one:

```
s = "J" + s[1:]    # 'Jython'
```

Advantage: Immutability makes strings thread-safe and usable as dictionary keys.

String Immutability: Concept and Performance

Key Concept: Strings are Immutable

Once created in memory, string objects cannot be modified in place.

```
s = "Python"
s[0] = "J"  # TypeError: 'str' object does not support item assignment
```

Creating New Strings

To change a string, construct a new one:

```
s = "J" + s[1:]  # 'Jython'
```

Advantage: Immutability makes strings thread-safe and usable as dictionary keys.

Computational Performance

Repeatedly appending strings in a loop with `s += char` creates a new copy each time ($O(N^2)$ complexity).

► **Best Practice:** Accumulate in a list and use `''.join(chars)` ($O(N)$).

String Formatting: Formatted String Literals (f-Strings)

Modern Formatting (PEP 498)

Introduced in Python 3.6, prefixing a string literal with `f` allows embedding arbitrary Python expressions directly inside curly braces `{}`.

```
1 name = "Alice"
2 age = 20
3 a, b = 42, 30
4
5 # Direct expression evaluation
6 msg = f"{name} is {age} years old."
7 # "Alice is 20 years old."
8
9 calc = f"GCD of {a} and {b} is {a-b}?"
10      "
```

String Formatting: Formatted String Literals (f-Strings)

Modern Formatting (PEP 498)

Introduced in Python 3.6, prefixing a string literal with `f` allows embedding arbitrary Python expressions directly inside curly braces `{}`.

```
1 name = "Alice"
2 age = 20
3 a, b = 42, 30
4
5 # Direct expression evaluation
6 msg = f"{name} is {age} years old."
7 # "Alice is 20 years old."
8
9 calc = f"GCD of {a} and {b} is {a-b}?"
10 "
```

Format Specifiers

```
1 import math
2
3 # Floating-point precision (
4   Assignment 1 Pi!)
5 f"Pi: {math.pi:.4f}"
6 # Output: 'Pi: 3.1416'
7
8 # Padding and alignment
9 f"{'ID':<6}: {'Score':>5}"
10 # Output: 'ID      : Score'
```


Structuring Code and Software Principles

Procedural Abstraction and Software Principles

A monolithic script works for one-off calculations, but real engineering systems demand structure, modularity, and maintainability.

Procedural Abstraction

A **function** packages a computational sequence into an isolated, named unit.

- ▶ Hides implementation complexity behind a clean interface.
- ▶ Separates *what* a computation does from *how* it is implemented.

Procedural Abstraction and Software Principles

A monolithic script works for one-off calculations, but real engineering systems demand structure, modularity, and maintainability.

Procedural Abstraction

A **function** packages a computational sequence into an isolated, named unit.

- ▶ Hides implementation complexity behind a clean interface.
- ▶ Separates *what* a computation does from *how* it is implemented.

Guiding Software Principles

- ▶ **Abstraction Principle (Pierce):** "Each significant piece of functionality should be implemented in just one place."

Procedural Abstraction and Software Principles

A monolithic script works for one-off calculations, but real engineering systems demand structure, modularity, and maintainability.

Procedural Abstraction

A **function** packages a computational sequence into an isolated, named unit.

- ▶ Hides implementation complexity behind a clean interface.
- ▶ Separates *what* a computation does from *how* it is implemented.

Guiding Software Principles

- ▶ **Abstraction Principle (Pierce):** "Each significant piece of functionality should be implemented in just one place."
- ▶ **DRY (Don't Repeat Yourself):** Duplicated code creates duplicated bugs. Reuse via functions.

Functions: Definition, Type Hints, and Reusability

Refactored Function

```
1 def gcd(a: int, b: int) -> int:
2     """Compute GCD of two positive
3         integers."""
4     while a != b:
5         if a > b:
6             a = a - b
7         else:
8             b = b - a
9     return a # Return value to caller!
```

Functions: Definition, Type Hints, and Reusability

Refactored Function

```
1 def gcd(a: int, b: int) -> int:
2     """Compute GCD of two positive
3         integers."""
4     while a != b:
5         if a > b:
6             a = a - b
7         else:
8             b = b - a
9     return a # Return value to caller!
```

Calling the Function

```
1 ans1 = gcd(42, 30)    # 6
2 ans2 = gcd(100, 15)   # 5
3 print(f"Results: {ans1}, {ans2}")
```

Functions: Definition, Type Hints, and Reusability

Refactored Function

```
1 def gcd(a: int, b: int) -> int:
2     """Compute GCD of two positive
3         integers."""
4     while a != b:
5         if a > b:
6             a = a - b
7         else:
8             b = b - a
9     return a # Return value to caller!
```

Calling the Function

```
1 ans1 = gcd(42, 30)    # 6
2 ans2 = gcd(100, 15)   # 5
3 print(f"Results: {ans1}, {ans2}")
```

Return vs. Print

A function should **return** its result so that other code can consume it. Printing inside functions destroys reusability!

Variable Scope: The LEGB Rule

What is Scope?

Scope defines where a name is visible and accessible. Local names created inside a function are destroyed when the function returns.

Python searches for names in LEGB order:

Variable Scope: The LEGB Rule

What is Scope?

Scope defines where a name is visible and accessible. Local names created inside a function are destroyed when the function returns.

Python searches for names in LEGB order:

L (Local): Inside the current function.

1. Local (L) (current function)

Variable Scope: The LEGB Rule

What is Scope?

Scope defines where a name is visible and accessible. Local names created inside a function are destroyed when the function returns.

Python searches for names in LEGB order:

L (Local): Inside the current function.

E (Enclosing): In enclosing functions (closures).

2. **Enclosing (E)** (enclosing functions)

1. **Local (L)** (current function)

Variable Scope: The LEGB Rule

What is Scope?

Scope defines where a name is visible and accessible. Local names created inside a function are destroyed when the function returns.

Python searches for names in LEGB order:

L (Local): Inside the current function.

E (Enclosing): In enclosing functions (closures).

G (Global): Module top-level variables.

1. **Local (L)** (current function)
2. **Enclosing (E)** (enclosing functions)
3. **Global (G)** (module top-level)

Variable Scope: The LEGB Rule

What is Scope?

Scope defines where a name is visible and accessible. Local names created inside a function are destroyed when the function returns.

Python searches for names in LEGB order:

L (Local): Inside the current function.

E (Enclosing): In enclosing functions (closures).

G (Global): Module top-level variables.

B (Built-in): Built-in names (`len`, `int`, `print`).

4. **Built-in (B)** (`print`, `len`, `int`)
3. **Global (G)** (module top-level)
2. **Enclosing (E)** (enclosing functions)
1. **Local (L)** (current function)

Variable Scope: The LEGB Rule

What is Scope?

Scope defines where a name is visible and accessible. Local names created inside a function are destroyed when the function returns.

Python searches for names in LEGB order:

L (Local): Inside the current function.

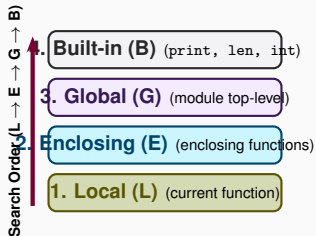
E (Enclosing): In enclosing functions (closures).

G (Global): Module top-level variables.

B (Built-in): Built-in names (`len`, `int`, `print`).

Design Rule

Avoid globals! Functions should communicate purely via explicit parameters and return values.



Defensive Programming: Enforcing Preconditions

The Problem: Invalid Inputs

What happens if someone calls `gcd(42, -5)`?

- ▶ $42 > -5 \implies a = 42 - (-5) = 47.$
- ▶ Next: $47 > -5 \implies a = 52, 57, \dots$ (**Infinite loop!**)

Defensive Programming: Enforcing Preconditions

The Problem: Invalid Inputs

What happens if someone calls `gcd(42, -5)`?

- ▶ $42 > -5 \implies a = 42 - (-5) = 47$.
- ▶ Next: $47 > -5 \implies a = 52, 57, \dots$ (**Infinite loop!**)

Fail Fast, Fail Loudly

Validate preconditions immediately to prevent runaway errors!

Enforcing Preconditions

```
1 def gcd(a: int, b: int) -> int:
2     # Check input types and range
3     if not (isinstance(a, int) and
4             isinstance(b, int)):
5         raise TypeError("Inputs must
6                           be integers")
7     if a <= 0 or b <= 0:
8         raise ValueError(f"Inputs
9                           must be > 0: {a}, {b}")
10    while a != b:
11        if a > b:
12            a -= b
13        else:
14            b -= a
15    return a
```

Robust Exception Handling: try / except / finally

Handling Errors Gracefully

```
1  try:
2      user_a = int(input("Enter a: "))
3      user_b = int(input("Enter b: "))
4      result = gcd(user_a, user_b)
5      print(f"GCD: {result}")
6  except ValueError as e:
7      print(f"Invalid input: {e}")
8  except TypeError as e:
9      print(f"Type error: {e}")
10 finally:
11     print("Execution complete.")
```


Robust Exception Handling: try / except / finally

Handling Errors Gracefully

```
1 try:
2     user_a = int(input("Enter a: "))
3     user_b = int(input("Enter b: "))
4     result = gcd(user_a, user_b)
5     print(f"GCD: {result}")
6 except ValueError as e:
7     print(f"Invalid input: {e}")
8 except TypeError as e:
9     print(f"Type error: {e}")
10 finally:
11     print("Execution complete.")
```

Architectural Principle: Detect Low, Handle High

- ▶ **Detect Low:** The low-level algorithm (gcd) detects the contract violation and raises an exception.
- ▶ **Handle High:** The user interface catches the exception and decides how to recover (e.g., prompt again).
- ▶ Do not catch exceptions unless you can meaningfully recover!

Verification and Testing with assert

Automated Unit Testing

```
1 def test_gcd():
2     """Unit tests for GCD function."""
3     assert gcd(42, 30) == 6, "Standard
4         failed"
5     assert gcd(30, 42) == 6, "
6         Commutativity failed"
7     assert gcd(17, 5) == 1, "Coprime
8         failed"
9     assert gcd(7, 7) == 7, "Identical
10        failed"
11    assert gcd(100, 10) == 10, "Multiple
12        failed"
13    print("All unit tests passed
14        successfully!")
15
16 test_gcd()
```

Verification and Testing with assert

Automated Unit Testing

```
1 def test_gcd():
2     """Unit tests for GCD function."""
3     assert gcd(42, 30) == 6, "Standard
4         failed"
5     assert gcd(30, 42) == 6, "
6         Commutativity failed"
7     assert gcd(17, 5) == 1, "Coprime
8         failed"
9     assert gcd(7, 7) == 7, "Identical
10        failed"
11     assert gcd(100, 10) == 10, "Multiple
12        failed"
13     print("All unit tests passed
14         successfully!")
15
16 test_gcd()
```

Testing Criteria (Kernighan & Pike)

- ▶ **Boundary Cases:** Smallest values (1, 1), coprime pairs (17, 5).
- ▶ **Symmetry:** $f(a, b) == f(b, a)$.
- ▶ **Code Coverage:** Exercise every branch of an if / else.

Verification and Testing with assert

Automated Unit Testing

```
1 def test_gcd():
2     """Unit tests for GCD function."""
3     assert gcd(42, 30) == 6, "Standard
4         failed"
5     assert gcd(30, 42) == 6, "
6         Commutativity failed"
7     assert gcd(17, 5) == 1, "Coprime
8         failed"
9     assert gcd(7, 7) == 7, "Identical
10        failed"
11     assert gcd(100, 10) == 10, "Multiple
12        failed"
13     print("All unit tests passed
14         successfully!")
15
16 test_gcd()
```

Testing Criteria (Kernighan & Pike)

- ▶ **Boundary Cases:** Smallest values (1, 1), coprime pairs (17, 5).
- ▶ **Symmetry:** $f(a, b) == f(b, a)$.
- ▶ **Code Coverage:** Exercise every branch of an if / else.

Coursework Note: Use assert statements to verify every exercise in Assignment 1!

Computational Challenge: Lift Control Systems

The Elevator Dispatch Problem

From Clean Algorithms to Real-World Stochastic Complexity

The Real-World Dilemma

In an 80-storey tower with 6,000 workers arriving during morning rush, naive elevator logic causes gridlock.

The Elevator Dispatch Problem

From Clean Algorithms to Real-World Stochastic Complexity

The Real-World Dilemma

In an 80-storey tower with 6,000 workers arriving during morning rush, naive elevator logic causes gridlock.

- **Stochastic Inputs:** Passenger arrivals and destination choices are dynamic and uncertain.

The Elevator Dispatch Problem

From Clean Algorithms to Real-World Stochastic Complexity

The Real-World Dilemma

In an 80-storey tower with 6,000 workers arriving during morning rush, naive elevator logic causes gridlock.

- ▶ **Stochastic Inputs:** Passenger arrivals and destination choices are dynamic and uncertain.
- ▶ **Physical Constraints:** Acceleration, speed, door cycles, and cabin capacity limits.

The Elevator Dispatch Problem

From Clean Algorithms to Real-World Stochastic Complexity

The Real-World Dilemma

In an 80-storey tower with 6,000 workers arriving during morning rush, naive elevator logic causes gridlock.

- ▶ **Stochastic Inputs:** Passenger arrivals and destination choices are dynamic and uncertain.
- ▶ **Physical Constraints:** Acceleration, speed, door cycles, and cabin capacity limits.
- ▶ **NP-Hard Problem:** Elevator routing is a dynamic Vehicle Routing Problem (VRP) with time windows.

The Elevator Dispatch Problem

From Clean Algorithms to Real-World Stochastic Complexity

The Real-World Dilemma

In an 80-storey tower with 6,000 workers arriving during morning rush, naive elevator logic causes gridlock.

- ▶ **Stochastic Inputs:** Passenger arrivals and destination choices are dynamic and uncertain.
- ▶ **Physical Constraints:** Acceleration, speed, door cycles, and cabin capacity limits.
- ▶ **NP-Hard Problem:** Elevator routing is a dynamic Vehicle Routing Problem (VRP) with time windows.

Performance Metrics (QoS)

AWT: Average Waiting Time (lobby to cab).

AJT: Average Journey Time (time inside cab).

Throughput: Passengers served per 5 min.

The Elevator Dispatch Problem

From Clean Algorithms to Real-World Stochastic Complexity

The Real-World Dilemma

In an 80-storey tower with 6,000 workers arriving during morning rush, naive elevator logic causes gridlock.

- ▶ **Stochastic Inputs:** Passenger arrivals and destination choices are dynamic and uncertain.
- ▶ **Physical Constraints:** Acceleration, speed, door cycles, and cabin capacity limits.
- ▶ **NP-Hard Problem:** Elevator routing is a dynamic Vehicle Routing Problem (VRP) with time windows.

Performance Metrics (QoS)

AWT: Average Waiting Time (lobby to cab).

AJT: Average Journey Time (time inside cab).

Throughput: Passengers served per 5 min.

The Inherent Trade-off

Stopping for an extra passenger reduces their **AWT**, but increases **AJT** for everyone inside!

Naive Dispatch Algorithms: FIFO and SSTF

1. First-In, First-Out (FIFO)

Serve calls strictly in order of arrival.

- ▶ **Advantage:** Perfectly fair; zero starvation.

Naive Dispatch Algorithms: FIFO and SSTF

1. First-In, First-Out (FIFO)

Serve calls strictly in order of arrival.

- ▶ **Advantage:** Perfectly fair; zero starvation.
- ▶ **Fatal Flaw:** Wild zig-zagging between distant floors; catastrophic travel times.

Naive Dispatch Algorithms: FIFO and SSTF

1. First-In, First-Out (FIFO)

Serve calls strictly in order of arrival.

- ▶ **Advantage:** Perfectly fair; zero starvation.
- ▶ **Fatal Flaw:** Wild zig-zagging between distant floors; catastrophic travel times.

2. Shortest Seek Time First (SSTF)

Always move to nearest outstanding call.

- ▶ **Advantage:** Minimises immediate transit distance.

Naive Dispatch Algorithms: FIFO and SSTF

1. First-In, First-Out (FIFO)

Serve calls strictly in order of arrival.

- ▶ **Advantage:** Perfectly fair; zero starvation.
- ▶ **Fatal Flaw:** Wild zig-zagging between distant floors; catastrophic travel times.

2. Shortest Seek Time First (SSTF)

Always move to nearest outstanding call.

- ▶ **Advantage:** Minimises immediate transit distance.
- ▶ **Fatal Flaw: Starvation!** Distant calls wait indefinitely while the lift bounces between middle floors.

Naive Dispatch Algorithms: FIFO and SSTF

1. First-In, First-Out (FIFO)

Serve calls strictly in order of arrival.

- ▶ **Advantage:** Perfectly fair; zero starvation.
- ▶ **Fatal Flaw:** Wild zig-zagging between distant floors; catastrophic travel times.

2. Shortest Seek Time First (SSTF)

Always move to nearest outstanding call.

- ▶ **Advantage:** Minimises immediate transit distance.
- ▶ **Fatal Flaw: Starvation!** Distant calls wait indefinitely while the lift bounces between middle floors.

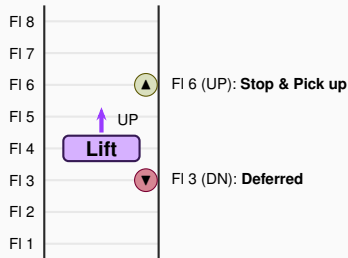
Conclusion: Pure fairness (FIFO) destroys throughput, while greedy proximity (SSTF) destroys fairness. We need structured directional sweeping!

Collective Control: The SCAN and LOOK Algorithms

SCAN (The Classic Elevator Algorithm)

The car travels continuously in one direction, serving all calls in that direction, until reaching the **end of the shaft**, then reverses.

- ▶ Batches calls in direction of travel.
- ▶ Bounded wait time $\implies$ prevents starvation!



Collective Control: The SCAN and LOOK Algorithms

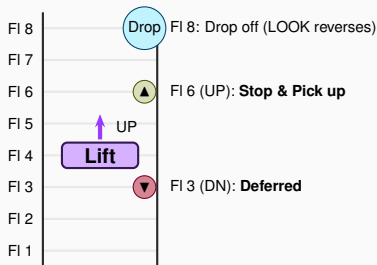
SCAN (The Classic Elevator Algorithm)

The car travels continuously in one direction, serving all calls in that direction, until reaching the **end of the shaft**, then reverses.

- ▶ Batches calls in direction of travel.
- ▶ Bounded wait time $\implies$ prevents starvation!

LOOK (The Smart Optimization)

Travels only to the **furthest call** in current direction, then reverses! Avoids empty trips to the roof.



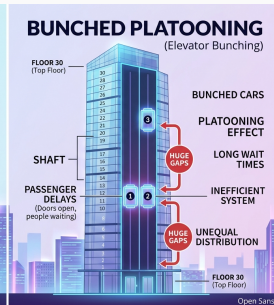
The System Instability: Elevator Bunching

Why Do Lifts Move in Packs, Just Like City Buses?

The Bunching Phenomenon

Without active coordination, multi-car elevator banks naturally degenerate into **bunched platoons**, causing massive gaps in service.

The Vicious Cycle:



The System Instability: Elevator Bunching

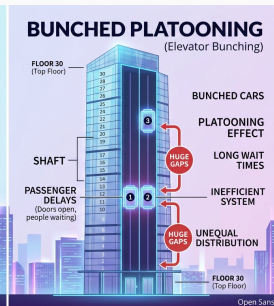
Why Do Lifts Move in Packs, Just Like City Buses?

The Bunching Phenomenon

Without active coordination, multi-car elevator banks naturally degenerate into **bunched platoons**, causing massive gaps in service.

The Vicious Cycle:

1. **Delay:** Lift A is delayed slightly (slow boarding).



The System Instability: Elevator Bunching

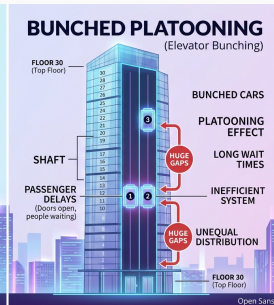
Why Do Lifts Move in Packs, Just Like City Buses?

The Bunching Phenomenon

Without active coordination, multi-car elevator banks naturally degenerate into **bunched platoons**, causing massive gaps in service.

The Vicious Cycle:

1. **Delay:** Lift A is delayed slightly (slow boarding).
2. **Gap Widens:** Time gap ahead of Lift A expands
⇒ more passengers arrive.



The System Instability: Elevator Bunching

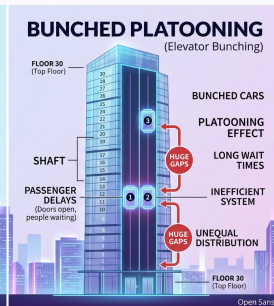
Why Do Lifts Move in Packs, Just Like City Buses?

The Bunching Phenomenon

Without active coordination, multi-car elevator banks naturally degenerate into **bunched platoons**, causing massive gaps in service.

The Vicious Cycle:

1. **Delay:** Lift A is delayed slightly (slow boarding).
2. **Gap Widens:** Time gap ahead of Lift A expands
⇒ more passengers arrive.
3. **Compounding Delay:** Lift A stops at every floor to load crowds.



The System Instability: Elevator Bunching

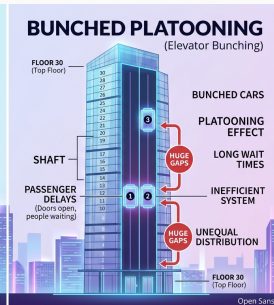
Why Do Lifts Move in Packs, Just Like City Buses?

The Bunching Phenomenon

Without active coordination, multi-car elevator banks naturally degenerate into **bunched platoons**, causing massive gaps in service.

The Vicious Cycle:

1. **Delay:** Lift A is delayed slightly (slow boarding).
2. **Gap Widens:** Time gap ahead of Lift A expands $\Rightarrow$ more passengers arrive.
3. **Compounding Delay:** Lift A stops at every floor to load crowds.
4. **Catch-up:** Trailing Lift B finds empty lobbies $\Rightarrow$ speeds up and catches Lift A!

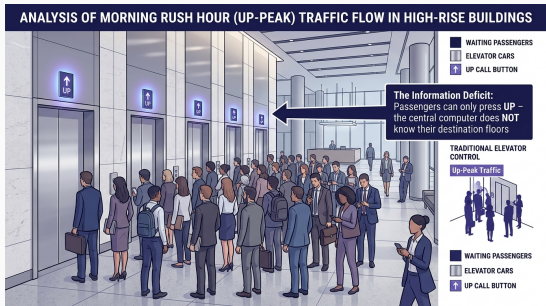


The Up-Peak Crisis and the Information Deficit

Up-Peak Traffic Pattern

During morning rush, traffic is heavily asymmetric:

- ▶ Nearly 100% of passengers enter at Ground Floor.
- ▶ Destinations scattered across Floors 2 to 60.



The Up-Peak Crisis and the Information Deficit

Up-Peak Traffic Pattern

During morning rush, traffic is heavily asymmetric:

- ▶ Nearly 100% of passengers enter at Ground Floor.
- ▶ Destinations scattered across Floors 2 to 60.

The "Information Deficit"

- ▶ In lobby, passengers press only [▲] (UP).
- ▶ Controller has **zero knowledge of destinations!**
- ▶ 16 passengers board Lift A → 14 distinct stops!

ANALYSIS OF MORNING RUSH HOUR (UP-PEAK) TRAFFIC FLOW IN HIGH-RISE BUILDINGS



The Revolution: Destination Dispatch Systems (DDS)

How Smart Information Architecture Transformed High-Rise Transit

A Paradigm Shift

Instead of pressing [▲] in the hallway:

1. Enters **destination floor** on lobby keypad *before* boarding.



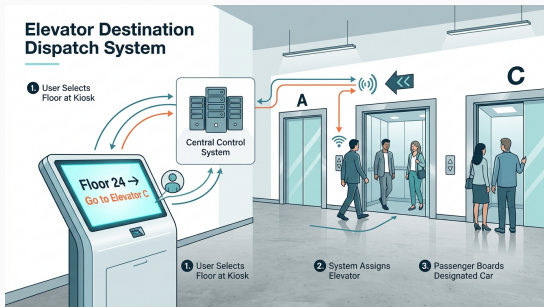
The Revolution: Destination Dispatch Systems (DDS)

How Smart Information Architecture Transformed High-Rise Transit

A Paradigm Shift

Instead of pressing [▲] in the hallway:

1. Enters **destination floor** on lobby keypad *before* boarding.
2. System immediately directs: "**Go to Car C**".



The Revolution: Destination Dispatch Systems (DDS)

How Smart Information Architecture Transformed High-Rise Transit

A Paradigm Shift

Instead of pressing [▲] in the hallway:

1. Enters **destination floor** on lobby keypad *before* boarding.
2. System immediately directs: **"Go to Car C"**.
3. Inside the elevator: **NO FLOOR BUTTONS!**



The Revolution: Destination Dispatch Systems (DDS)

How Smart Information Architecture Transformed High-Rise Transit

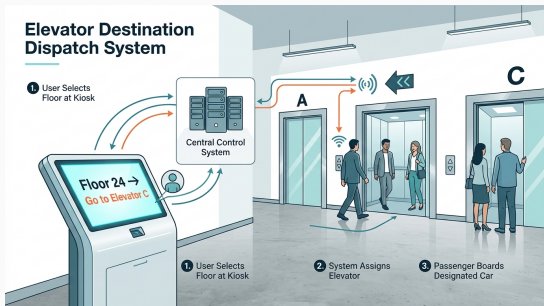
A Paradigm Shift

Instead of pressing [▲] in the hallway:

1. Enters **destination floor** on lobby keypad *before* boarding.
2. System immediately directs: "**Go to Car C**".
3. Inside the elevator: **NO FLOOR BUTTONS!**

The Power of Passenger Grouping

- ▶ **Car A:** Dispatched to Floors 10–18.
- ▶ **Car B:** Express to Floors 19–28.
- ▶ **Car C:** Express to Floors 29–40.



Why DDS Outperforms Traditional Systems

Traditional: Chaotic Local Bus

- ▶ Passengers board any arriving car.
- ▶ 8–12 stops on an up-trip.
- ▶ High round-trip time (≈ 150 s).
- ▶ Massive lobby queues in rush hour.

Why DDS Outperforms Traditional Systems

Traditional: Chaotic Local Bus

- ▶ Passengers board any arriving car.
- ▶ 8–12 stops on an up-trip.
- ▶ High round-trip time (≈ 150 s).
- ▶ Massive lobby queues in rush hour.

DDS: Coordinated Express Service

- ▶ Batches co-destined passengers.
- ▶ Only 2–4 stops per trip!
- ▶ Round-trip time cut by $\approx 50\%$.
- ▶ **Throughput jumps by 30%+** without new shafts.

Why DDS Outperforms Traditional Systems

Traditional: Chaotic Local Bus

- ▶ Passengers board any arriving car.
- ▶ 8–12 stops on an up-trip.
- ▶ High round-trip time (≈ 150 s).
- ▶ Massive lobby queues in rush hour.

DDS: Coordinated Express Service

- ▶ Batches co-destined passengers.
- ▶ Only 2–4 stops per trip!
- ▶ Round-trip time cut by $\approx 50\%$.
- ▶ **Throughput jumps by 30%+** without new shafts.

The Human Psychology of DDS

- ▶ **Initial Disorientation:** Entering an elevator with no buttons creates panic!
- ▶ **Loss of Perceived Control:** Users feel the system has taken over their choice.
- ▶ **The Reality:** Passengers reach their destination faster, even if the wait in the lobby is slightly longer.

Why DDS Outperforms Traditional Systems

Traditional: Chaotic Local Bus

- ▶ Passengers board any arriving car.
- ▶ 8–12 stops on an up-trip.
- ▶ High round-trip time (≈ 150 s).
- ▶ Massive lobby queues in rush hour.

DDS: Coordinated Express Service

- ▶ Batches co-destined passengers.
- ▶ Only 2–4 stops per trip!
- ▶ Round-trip time cut by $\approx 50\%$.
- ▶ **Throughput jumps by 30%+** without new shafts.

The Human Psychology of DDS

- ▶ **Initial Disorientation:** Entering an elevator with no buttons creates panic!
- ▶ **Loss of Perceived Control:** Users feel the system has taken over their choice.
- ▶ **The Reality:** Passengers reach their destination faster, even if the wait in the lobby is slightly longer.

Algorithms in DDS

Real-time assignment uses **Genetic Algorithms (GAs)** and heuristic branch-and-bound.



Modern Frontier: Machine Learning and Reinforcement Learning

1. Predictive Traffic Modelling

Buildings follow predictable temporal rhythms:

- ▶ Morning up-peak (inbound).
- ▶ Lunchtime two-way traffic (inter-floor).
- ▶ Evening down-peak (outbound).

Deep learning models predict Origin-Destination (OD) traffic matrices 15 minutes ahead, allowing controllers to **pre-park idle cars** at high-demand floors.

Modern Frontier: Machine Learning and Reinforcement Learning

1. Predictive Traffic Modelling

Buildings follow predictable temporal rhythms:

- ▶ Morning up-peak (inbound).
- ▶ Lunchtime two-way traffic (inter-floor).
- ▶ Evening down-peak (outbound).

Deep learning models predict Origin-Destination (OD) traffic matrices 15 minutes ahead, allowing controllers to **pre-park idle cars** at high-demand floors.

2. Reinforcement Learning (RL)

Formulated as a Markov Decision Process (MDP):

- ▶ **State** s_t : Positions, velocities, door states, loads, registered calls.
- ▶ **Action** a_t : Assign call to Lift i ; or dispatch idle lift to zone z .
- ▶ **Reward** r_t : Penalty for passenger wait time $AWT + \lambda \cdot AJT + \mu \cdot \text{Energy}$.

RL agents learn non-linear dispatch policies that adapt to irregular traffic spikes.



Summary and Key Takeaways

Foundations of Programming

- ▶ **Algorithms:** Precise sequences verified by invariants; mathematical proofs ensure correctness and termination.
- ▶ **Python Syntax:** Dynamic typing with strong enforcement; arbitrary precision integers; division subtleties (`/`, `//`, `%`).
- ▶ **Strings:** Immutable Unicode sequences; zero-based indexing and slicing; f-strings.
- ▶ **Modularity:** Functions as procedural abstractions; LEGB variable scoping.
- ▶ **Defensive Coding:** Assertions for unit testing; fail-fast exceptions.

Summary and Key Takeaways

Foundations of Programming

- ▶ **Algorithms:** Precise sequences verified by invariants; mathematical proofs ensure correctness and termination.
- ▶ **Python Syntax:** Dynamic typing with strong enforcement; arbitrary precision integers; division subtleties (`/`, `//`, `%`).
- ▶ **Strings:** Immutable Unicode sequences; zero-based indexing and slicing; f-strings.
- ▶ **Modularity:** Functions as procedural abstractions; LEGB variable scoping.
- ▶ **Defensive Coding:** Assertions for unit testing; fail-fast exceptions.

Computational Systems Engineering

- ▶ **Real-World Optimization:** Complex stochastic problems (NP-hard) require balancing competing QoS metrics (AWT vs. AJT).
- ▶ **Elevator Bunching:** Unregulated multi-agent systems naturally degenerate into bunched platoons.
- ▶ **Information Architecture:** Destination Dispatch Systems (DDS) solve the information deficit, cutting stops and boosting throughput by 30%+.
- ▶ **Modern AI:** Moving from fixed heuristics (SCAN/LOOK) to predictive ML and adaptive RL.